

Chandra: Entities and EJB

Domain model, Hibernate, the session beans, the queue and the audit trail. Campus Placement and Training Cell.

1. Your role in one sentence

You own the domain model and the business logic: the six JPA entities that Hibernate maps onto MySQL, and the EJBs that read and write them, decide who is eligible, queue notifications and record the audit trail.

2. What you own

FILE	LINES	WHAT IT IS
entity/AppUser.java	158	Account and credentials
entity/StudentProfile.java	155	Academic record, one to one
entity/Company.java	105	Recruiter, @OneToMany to drives
entity/Drive.java	207	Drive and its eligibility inputs
entity/JobApplication.java	133	Student to drive join, unique key
entity/AuditLog.java	102	One audit row
entity/Role, ApplicationStatus, DriveStatus	20, 28, 22	The three enums
META-INF/persistence.xml	68	Persistence unit, Hibernate provider
ejb/ApplicationService.java	171	Stateless: apply, status, withdraw
ejb/DriveService.java	153	Stateless: drives, companies, check()
ejb/StudentService.java	109	Stateless: profiles, search
ejb/EligibilityRules, Eligibility	69, 46	The rules, and the verdict object
ejb/ShortlistBean.java	136	Stateful shortlist basket
ejb/NotificationSender.java	56	JMS producer, declares the queue
ejb/NotificationMDB.java	58	Message driven consumer
ejb/PortalStatsBean.java	157	Singleton: counters, JNDI report

FILE	LINES	WHAT IT IS
ejb/AuditInterceptor.java	161	@AroundInvoke audit interceptor
ejb/AuditWriter, AuditService	30, 42	Write one row, read the rows
test/ejb/EligibilityRulesTest.java	179	10 tests, no database

Not yours: `AuthService` and `AuthResult` (Tiya), `DataSeeder` (a teammate), `MockScoreBean` (Yuvraj, also stateful). `EligibilityRules` and `Eligibility` are plain classes, not beans. `ShortlistServlet` is Karan's but drives your `ShortlistBean`. Your interceptor also wraps `AuthService`, which is why sign ins show on the Audit page.

3. The 60 second explanation

SAY THIS

My part is the model and the business rules. Six JPA entities, mapped by Hibernate onto MySQL. An application row joins a student to a drive, with a unique key on the pair so nobody can apply twice.

The rules live in EJBs. The three services are stateless and the container wraps each write in a transaction. The shortlist basket is stateful, one per officer. A singleton starts at deploy. When an application changes, a message goes on a JMS queue and a message driven bean records it. An interceptor writes an audit row for every change, with passwords redacted. Eligibility is one plain method that lists every reason a student fails, so I can show it live.

4. An EJB cheat table

BEAN	TYPE	ANNOTATIONS	PURPOSE	USED BY
ApplicationService	Stateless	@Stateless @Interceptors	Apply, status, withdraw	Student, officer servlets
DriveService	Stateless	same	Drives, <code>check()</code>	Drive pages
StudentService	Stateless	same	Profiles, search	Profile pages
NotificationSender	Stateless	@JMSDestinationDefinition	Queue a message	Services, basket

BEAN	TYPE	ANNOTATIONS	PURPOSE	USED BY
NotificationMDB	Message driven	@MessageDriven	Consume, save a row	Container only
ShortlistBean	Stateful	@Stateful @Remove	Officer's basket	ShortlistServlet
PortalStatsBean	Singleton	@Singleton @Startup	Counters, JNDI	Login, apply, System

BEAN	TYPE	ANNOTATIONS	PURPOSE	USED BY
AuditInterceptor	Interceptor	@AroundInvoke	Audit row per call	Three services, AuthService
AuditWriter	Stateless	REQUIRES_NEW	Save audit row	The interceptor
AuditService	Stateless	SUPPORTS	Read audit rows	Audit page

Transactions and rollback

- **SUPPORTS** : every read in the three services and all of `AuditService` . Join a transaction if there is one, never start one.
- **REQUIRED** (default, no annotation): every write (`apply` , `updateStatus` , `withdraw` , `save` , `delete` , `ShortlistBean.commit`) and the MDB. Written out on `NotificationSender.send` .
- **REQUIRES_NEW** : `AuditWriter.write` , so the audit row commits even if the caller rolls back. **MANDATORY** and **NEVER** are not used, and there is no `UserTransaction` anywhere.

What a rollback means. The container starts the transaction before a business method and commits when it returns. A runtime exception is a system exception: the transaction rolls back, so the INSERT or UPDATE from that call never happens, and the caller receives the original wrapped in `EJBException` . That is why `ApplicationServlet.withdraw` catches `EJBException` and reads `getCause()` . `ShortlistBean.commit()` therefore moves every ticked candidate or none. JMS errors are swallowed on purpose, so a broken queue never undoes an edit.

JNDI, and @EJB compared with a lookup

- `java:app/jdbc/placementDS` : the pooled datasource, named in `persistence.xml` ; `PortalStatsBean` gets it by `@Resource` and by `InitialContext.lookup` .
- `java:module/ShortlistBean` : `new InitialContext().lookup(...)` , `ShortlistServlet` line 135. The queue and its factory are `java:app/jms/...` names, created by annotations on `NotificationSender` .
- **@EJB** fills a field once when the object is created, right for stateless beans. A **lookup** asks when you need it and gives a new stateful instance each time, so `ShortlistBean` is looked up and kept in the `HttpSession`.

JPA mapping

- **Foreign keys.** The owning side has `@JoinColumn` and its table holds the key; the other side says `mappedBy` . `@OneToOne` `StudentProfile.user` owns `user_id` (unique). `@ManyToOne` `Drive.company`

owns `company_id`, inverse `Company.drives ( @OneToMany )`. `@ManyToOne` `JobApplication.student` and `.drive` own `student_id` and `drive_id`. All four exist in MySQL.

- **No double application.** `JobApplication.java` line 26: `@UniqueConstraint(name = "uk_student_drive", columnNames = {"student_id", "drive_id"})`. `hasApplied` is the polite check; the database rule catches a replayed POST or a race.
- **EAGER and LAZY.** The three `@ManyToOne` links and the `@OneToOne` are EAGER, so a page can print `application.student.user.fullName` after the transaction ended. The three `@OneToMany` lists are LAZY and throw `LazyInitializationException` on a detached entity, so the beans count with queries instead.
- **Named queries.** 16 are defined (for example `JobApplication.exists`). Three are never called: `StudentProfile.byRoll`, `Company.byName`, `Drive.byCompany`.

persistence.xml

```
<persistence-unit name="placementPU" transaction-type="JTA">           line 18
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider> line 19
  <jta-data-source>java:app/jdbc/placementDS</jta-data-source>         line 26
  hibernate.hbm2ddl.auto = update                                       line 47
```

- Hibernate 6.4.4 (the server log says "Hibernate ORM core version 6.4.4.Final") does the mapping, not whatever the server bundles. JTA: GlassFish owns the transaction, Hibernate joins it. `update` adds missing tables on deploy and never drops data; `create-drop` would wipe it.

Entity lifecycle with a real example

- **Transient:** `new JobApplication(...)`, `ApplicationService.java` line 79.
- **Managed:** after `em.persist` (line 80) or `em.find` in `updateStatus` (line 88). `setStatus` on a managed entity is enough: the UPDATE runs at commit.
- **Detached:** anything a bean returns, because the transaction is over. `driveService.find()` runs with no transaction, so the servlet gets a detached drive. `DriveService.save` uses `merge` for one with an id.
- **Removed:** `em.remove(application)` in `withdraw`, line 119. The DELETE runs at commit.

The eligibility engine, step by step

`EligibilityRules.check(student, drive, alreadyApplied, today)`, line 32. A null student or drive is rejected at once (line 36). Otherwise it never stops at the first failure: every failing rule adds a reason. `DriveService.check()` (line 148) asks the database one question, `hasApplied`, then calls it with `LocalDate.now()`.

- **1 Drive status** (41): not OPEN. "This drive has been closed by the placement cell."
- **2 Deadline** (44): last date before today, the last day itself passes. "The last date to apply was 2026-08-29."
- **3 CGPA** (47): below the minimum, equal passes. "Requires a CGPA of 7.50, your record shows 5.60."
- **4 Backlogs** (52): above the limit, equal passes. "Allows at most 0 backlog(s), your record shows 3."
- **5 Branch** (57): the drive lists branches and none matches, ignoring case. Empty list means any. "Open to Mechanical, Civil, your branch is Computer Science."
- **6 Already applied** (64): a flag, not a rejection. `isEligible()` stays true, `isCanApply()` is false.

CAREFUL

There is no "already placed" rule. A student holding an offer can still apply. Placement status only feeds counts and displays (`countPlaced` , `placedStudentIds`). If asked, say so.

It also runs inside the POST in `ApplicationServlet` (line 90). `EligibilityRulesTest` has 10 tests; the whole suite is 146.

The audit trail

- `@AroundInvoke record()` wraps each method of a bean with `@Interceptors : ctx.proceed()` (line 52), times it, hands the row to `AuditWriter` .
- **Reads are skipped.** A name starting `has`, `for`, `list`, `find`, `count`, `placed`, `statusBreakdown`, `search`, `check` or ending `Taken` writes no row on success (lines 31, 69). A failing read is still recorded.
- **Entities print as Type#id** (`label()` , line 117), for example `StudentProfile#1`, `Drive#8` for `apply` . Strings, numbers and enums print as themselves.
- **Secrets are redacted:** a parameter named `password`, `secret`, `token` or `credential` prints `[redacted]` (line 131).
- The actor is always `portal` (line 149): sign in is by the app's own filter, so the audit does not record which officer acted.

The message driven bean path

- **Message and sender:** a text string such as "Aarti Deshpande moved from Applied to Shortlisted for Lumen Health", sent by `NotificationSender.send()` from `apply` , `updateStatus` , `withdraw` and `ShortListBean.commit` .
- **Consumer:** `NotificationMDB.onMessage()` (line 43) logs "Notification delivered: ..." and saves `AuditLog("notifier", "NOTIFICATION", body)` , after the officer's request has returned.
- **Proof:** Audit page, search `NOTIFICATION` (actor `notifier`), or search `server\glassfish7\glassfish\domains\domain1\logs\server.log` for `Notification delivered` .

5. Code walk in VS Code

1 `persistence.xml` 18 to 47; `JobApplication.java` 26, 49 to 54; `StudentProfile.java` 39, 65.
Say: "Hibernate is the provider. This side owns the foreign keys, the unique key stops a double application, lists are lazy."

2 `ApplicationService.java` lines 24 to 32, `apply` 75, `updateStatus` 87. Say: "No begin or commit. The container makes the transaction, Hibernate saves lines 93 and 94 at commit, and a message is queued." Then `DriveService.java` 148, `EligibilityRules.java` 32 to 66, `EligibilityRulesTest.java` 139: "A static method, so tests need no server."

3 `NotificationSender.java` 26 to 51; `NotificationMDB.java` 27 to 52; `AuditInterceptor.java` 31, 45 to 57, 69, 117; `AuditWriter.java` 26. Say: "The sender creates its own queue, nobody calls the MDB, the container does. `@AroundInvoke` wraps the call, reads are skipped, entities print as `Type#id`, and `REQUIRES_NEW` lets the audit row survive a rollback."

4 `ShortlistBean.java` 35, 50, 52 to 60, 108, 131; `ShortlistServlet.java` 135 to 136, 99, 152; `PortalStatsBean.java` 33, 34, 45, 84. Say: "Stateful, one per officer, looked up by JNDI and kept in the `HttpSession`. Commit is one transaction, `@Remove` ends it. The singleton starts at deploy."

6. Live demo plan

Base `http://localhost:8080/placement`. Every password is `Campus@2026`. Officer in one browser, student in a private window. Seed: 25 students, 14 drives, 85 applications, 6 offers (Aarti holds none).

1 **Not eligible, with reasons.** Sign in as `aarti.deshpande@campus.edu` (Computer Science, CGPA 8.7, 0 backlogs). Open `/app/drive?id=9` (Ironbridge): branch No, "Open to Mechanical, Civil, your branch is Computer Science." Then `/app/drive?id=13` (Orbit, closed): three reasons at once. As `rohan.kapoor@campus.edu` (CGPA 5.6, 3 backlogs) open `/app/drive?id=4`: CGPA and backlog reasons.

2 **Officer moves a status, audit shows it.** Sign in as `tpo@campus.edu`, open **Pipeline** (`/admin/applications`), find Aarti Deshpande, Lumen Health, Applied (application 8), choose Shortlisted, add a remark, **Save**. Open **Audit** (`/admin/audit`): two rows, `ApplicationService.updateStatus` with detail 8, SHORTLISTED, your remark, then actor `notifier`, NOTIFICATION, "Aarti Deshpande moved from Applied to Shortlisted for Lumen Health". Note the [redacted] rows and that browsing added none. Search `server.log` for `Notification delivered`. Then set application 8 back to Applied.

3 **Withdraw refused, the exception story.** As Aarti open **My applications** (`/app/applications`). Withdraw only shows on Applied rows. Press F12, change the hidden `applicationId` in an Applied row's Withdraw form to 1 (Shortlisted), then click Withdraw and accept the dialog. The flash says the placement cell has already progressed it, so it cannot be withdrawn; the row is unchanged. Audit shows `ApplicationService.withdraw`, 1, 1 [failed: `IllegalStateException`]. Say: "The bean threw, the container rolled back and wrapped it in `EJBException`, and the servlet reads `getCause()`." **Click without editing the id and that application is deleted.**

4 **The stateful basket.** Officer, **Shortlist** (`/admin/shortlist`): pick a drive, tick two names, reload (ticks stay, nothing written), click **End the session**. `server.log` shows "ShortlistBean created" and "finish() called".

5 MySQL, joined tables, unique key. In PowerShell:

```
$env:MYSQL_PWD = 'Placement@2026'
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user placement_ce`

SELECT a.id, u.full_name, c.name AS company, d.job_role, a.status
FROM job_application a
JOIN student_profile s ON s.id = a.student_id
JOIN app_user u      ON u.id = s.user_id
JOIN drive d         ON d.id = a.drive_id
JOIN company c       ON c.id = d.company_id
WHERE a.id = 8;
```

Say: "Four tables, four foreign keys, built by Hibernate from the annotations." `SHOW CREATE TABLE job_application\G` shows `UNIQUE KEY uk_student_drive (student_id, drive_id)`. MySQL times are UTC, 5 h 30 min behind the pages. JNDI is on `/admin/system`.

7. Questions you will be asked

Stateless versus stateful session bean?

Stateless keeps nothing between calls, so any pooled instance serves any caller (the three services). Stateful is one instance per client that remembers its fields (`ShortlistBean`).

Why is the shortlist bean stateful, and where is it held?

The officer ticks names across page loads, so the selection belongs to one person. The instance lives in the EJB container (30 minute timeout), the reference in that officer's `HttpSession`. A servlet field would be shared by all officers.

Singleton bean and `@Startup`? Interceptor?

A singleton is one instance for the whole app; `@Startup` creates it at deploy. An interceptor's `@AroundInvoke` method wraps a business method and calls `ctx.proceed()`, so the beans hold no audit code.

What is JNDI, and how does `@EJB` differ from a lookup?

JNDI is the container's name service: look up a name, get a resource, no URL or password in code. `@EJB` looks up once at creation; an explicit lookup gives a new stateful instance each time.

What does @PersistenceContext do?

It injects a container managed, transaction scoped `EntityManager` for `placementPU`. You never open or commit it.

Why Hibernate? JPA versus Hibernate?

JPA is the specification, Hibernate the implementation, named in `persistence.xml`. It writes the SQL, so only the dialect changes for another database.

Which side owns the foreign key, and how is a double application stopped?

The side with `@JoinColumn`, for example `JobApplication.student`. `hasApplied` checks first, and the unique key `uk_student_drive` refuses a second row even if the check is bypassed.

LAZY versus EAGER, and what is a detached entity?

EAGER loads the related row with the entity, LAZY waits until touched. A detached entity has lost its transaction, as in a servlet; a lazy list on it throws `LazyInitializationException`. `merge` reattaches it.

What happens when an exception is thrown inside a transaction?

A runtime exception is a system exception: the container rolls the transaction back and the caller gets it wrapped in `EJBException`, so the servlet reads `getCause()`, as `withdraw` does. The audit row survives (`REQUIRES_NEW`). Reads use `SUPPORTS`, writes `REQUIRED`.

How does the MDB work, and can a placed student apply again?

`NotificationSender.send` queues a text message; the container calls `NotificationMDB.onMessage`, which logs it and saves a NOTIFICATION row. A placed student can apply again: no rule forbids it.

8. If something goes wrong

SYMPTOM	CAUSE AND WHAT TO DO
No NOTIFICATION row	The MDB runs a moment later. Refresh, or check <code>server.log</code> .
Withdrew a real row	You clicked before editing the id. Tell Karan.
Server not answering	Tell Karan. Restarting never wipes data.